Módulo 6

Estructuras de Control

Curso: TypeScript

60 min

Objetivos de Aprendizaje

- Implementar **condicionales** con tipos seguros
- Utilizar **tipos literals** para control de flujo
- Trabajar con **type guards**
- Aplicar **estructuras iterativas** con tipado adecuado

1. Contexto

Control de Flujo con Tipos

El Problema

En JavaScript, el control de flujo puede ser impredecible:

```
function processValue(value) {
  if (value.type === "user") {
    // ¿Qué propiedades tiene value aquí?
    console.log(value.name);
  }
}
```

La Solución con TypeScript

TypeScript añade seguridad al control de flujo:

```
type User = { type: "user"; name: string };
type Product = { type: "product"; price: number };

function processValue(value: User | Product) {
  if (value.type === "user") {
    // TypeScript sabe que es User
    console.log(value.name); // ✓
  } else {
    // TypeScript sabe que es Product
    console.log(value.price); // ✓
  }
}
```

2. Type Guards

Type Guards

Los type guards permiten reducir tipos dentro de condicionales:

```
// typeof para primitivos
function print(value: string | number) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  } else {
    console.log(value.toFixed(2));
  }
}

// instanceof para clases
class Dog { bark() {} }
class Cat { meow() {} }

function makeSound(pet: Dog | Cat) {
  if (pet instanceof Dog) {
    pet.bark();
  } else {
    pet.meow();
  }
```

Lab: Filtrar Tareas con Type Guards

Objetivo: Implementar filtros seguros para el sistema de tareas.

1. Define los tipos TaskStatus y TaskPriority
2. Crea un type guard para verificar tareas completadas
3. Crea un type guard para tareas asignadas
4. Implementa la función de filtrado

Código del Lab

```typescript
type TaskStatus = "pending" | "inProgress" | "completed";
type TaskPriority = "low" | "medium" | "high";

interface Task {
  id: string;
  title: string;
  status: TaskStatus;
  priority: TaskPriority;
  assigneeId?: string;
}

function isCompleted(task: Task): boolean {
  return task.status === "completed";
}

function isAssigned(task: Task): task is Task & { assigneeId: string } {
  return task.assigneeId !== undefined;
}
```

3. Switch con Literals

Switch y Tipos Literales

Los tipos literales permiten exhaustiveness checking:

```
type Status = "pending" | "approved" | "rejected";

function getStatusColor(status: Status): string {
  switch (status) {
    case "pending":
      return "yellow";
    case "approved":
      return "green";
    case "rejected":
      return "red";
    default:
      // Exhaustiveness check
      const _exhaustive: never = status;
      throw new Error(`Unknown status: ${status}`);
  }
}
```

Lab: Sistema de Prioridades

Objetivo: Implementar lógica de priorización con switch exhaustivo.

1. Crea la función de prioridad con switch
2. Añade exhaustiveness check
3. Crea función para obtener descripción

3. Iteraciones Seguras

Bucles con Tipos

Los arrays mantienen sus tipos en las iteraciones:

```
const tasks = [
  { id: "1", title: "Task 1", completed: false },
  { id: "2", title: "Task 2", completed: true }
];

// forEach con tipado correcto
tasks.forEach((task, index) => {
  console.log(`${index}: ${task.title}`);
});

// map retorna array del mismo tipo
const titles: string[] = tasks.map(t => t.title);

// filter mantiene el tipo
const completed = tasks.filter(t => t.completed);
```

Lab: Procesamiento de Colecciones

Objetivo: Procesar arrays de tareas con tipos seguros.

1. Implementa función para contar por estado
2. Implementa ordenación por prioridad
3. Implementa группировку (agrupación) por estado

```
// countByStatus
function countByStatus(tasks: Task[]): Record<TaskStatus, number> {
  const counts: Record<TaskStatus, number> = {
    pending: 0,
    inProgress: 0,
    completed: 0
  };
  tasks.forEach(task => {
    counts[task.status]++;
  });
  return counts;
}
```

Resumen

Puntos Clave

- **typeof** para primitivos
- **instanceof** para clases
- **in** para propiedades de objetos
- Exhaustiveness checking con **never**
- Type guards reducen tipos dentro de condicionales

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Aplicar type guards al sistema de biblioteca.

1. Crea type guards para tipos de miembro
2. Implementa switch exhaustivo para estados de préstamo
3. Filtra préstamos por tipo de miembro

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- `typeof` para primitivos, `instanceof` para objetos
- `in` verifica propiedades en objetos
- Switch con default permite exhaustiveness checking

⚠️ Gotchas

- Type narrowing no funciona en callbacks asíncronos
- Arrays tipados pueden perder tipo en `.map()`
- Verificar `null` / `undefined` explícitamente

Referencias

Recursos Adicionales

- Type Narrowing
- Literal Types

Gracias!

Módulo 6 Completado

Siguiente: Módulo 7 - Funciones y Tipado